| | |
|---|---|
| **Course Title**: Cryptography and Network Security Lab | **Course Code**: BCSE309P |
| **Faculty**: Prof. Karthika V | Slot: *L15+L16* |
| Regno: *23BCE1099* | Name: *Arkita Barua* |

Ex3

AES

Code

Sender

```
import java.io.*;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Scanner;

public class aes_client_sender {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in, "UTF-8");
        System.out.print("Plaintext: ");
        String plaintext = sc.nextLine();
        System.out.print("Key: ");
        String key = sc.nextLine();

        String cipherStr = AES.encryptToString(plaintext, key);

        try (Socket s = new Socket("localhost", 5000);
             DataOutputStream dos = new DataOutputStream(s.getOutputStream());
             DataInputStream dis = new DataInputStream(s.getInputStream())) {

            byte[] cipherUtf8 = cipherStr.getBytes(StandardCharsets.UTF_8);
            dos.writeInt(cipherUtf8.length);
            dos.write(cipherUtf8);
            dos.flush();

            System.out.println("Cypher text sent: "+cipherStr);

            int respLen = dis.readInt();
            byte[] resp = new byte[respLen];
            dis.readFully(resp);
```

```
            System.out.println("Receiver response: " + new String(resp,
StandardCharsets.UTF_8));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // AES (encrypt-only)
    static class AES {
        private static final int Nr = 10;
        private static final int[] SBOX = {
            0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0
xd7,0xab,0x76,
            0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0
xa4,0x72,0xc0,
            0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0
xd8,0x31,0x15,
            0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0
x27,0xb2,0x75,
            0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0
xe3,0x2f,0x84,
            0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0
x4c,0x58,0xcf,
            0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0
x3c,0x9f,0xa8,
            0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0
xff,0xf3,0xd2,
            0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0
x5d,0x19,0x73,
            0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0
x5e,0x0b,0xdb,
            0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0
x95,0xe4,0x79,
            0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0
x7a,0xae,0x08,
            0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0
xbd,0x8b,0x8a,
            0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0
xc1,0x1d,0x9e,
            0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0
x55,0x28,0xdf,
            0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0
x54,0xbb,0x16
        };
        private static final int[] RCON =
{0x00,0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0x1B,0x36};
```

```
        private static String bytesToHex(byte[] b) { StringBuilder sb=new
StringBuilder(); for(byte x:b) sb.append(String.format("%02X", x&0xFF));
return sb.toString(); }
        private static String stateToHex(byte[][] s) { StringBuilder sb=new
StringBuilder(); for(int r=0;r<4;r++){ for(int c=0;c<4;c++)
sb.append(String.format("%02X", s[r][c]&0xFF)); if(r<3) sb.append(' ');}
return sb.toString(); }
        private static byte[][] bytesToState(byte[] in){ byte[][] s=new
byte[4][4]; for(int i=0;i<16;i++) s[i%4][i/4]=in[i]; return s; }
        private static byte[] stateToBytes(byte[][] state){ byte[] out=new
byte[16]; for(int i=0;i<16;i++) out[i]=state[i%4][i/4]; return out; }

        private static int mul(int a,int b){ int res=0; int aa=a&0xFF,
bb=b&0xFF; for(int i=0;i<8;i++){ if((bb&1)!=0) res^=aa; boolean
hi=(aa&0x80)!=0; aa=(aa<<1)&0xFF; if(hi) aa^=0x1B; bb>>>=1; } return res; }

        private static void subBytes(byte[][] s){ for(int r=0;r<4;r++) for(int
c=0;c<4;c++) s[r][c]=(byte)SBOX[s[r][c]&0xFF]; }
        private static void shiftRows(byte[][] s){ for(int r=1;r<4;r++){
byte[] tmp=new byte[4]; for(int c=0;c<4;c++) tmp[c]=s[r][(c+r)%4]; for(int
c=0;c<4;c++) s[r][c]=tmp[c]; } }
        private static void mixColumns(byte[][] s){ for(int c=0;c<4;c++){ int
a0=s[0][c]&0xFF,a1=s[1][c]&0xFF,a2=s[2][c]&0xFF,a3=s[3][c]&0xFF; int
r0=mul(0x02,a0)^mul(0x03,a1)^a2^a3; int r1=a0^mul(0x02,a1)^mul(0x03,a2)^a3;
int r2=a0^a1^mul(0x02,a2)^mul(0x03,a3); int
r3=mul(0x03,a0)^a1^a2^mul(0x02,a3); s[0][c]=(byte)r0; s[1][c]=(byte)r1;
s[2][c]=(byte)r2; s[3][c]=(byte)r3; } }
        private static void addRoundKey(byte[][] s, byte[] rk){ for(int
c=0;c<4;c++) for(int r=0;r<4;r++)
s[r][c]=(byte)((s[r][c]&0xFF)^(rk[c*4+r]&0xFF)); }

        private static byte[] expandKey(byte[] key){
            byte[] expanded=new byte[16*(Nr+1)];
System.arraycopy(key,0,expanded,0,16);
            int bytesGenerated=16; int rconIter=1; byte[] temp=new byte[4];
            while(bytesGenerated<expanded.length){
                for(int i=0;i<4;i++) temp[i]=expanded[bytesGenerated-4+i];
                if(bytesGenerated%16==0){
                    byte t=temp[0]; temp[0]=temp[1]; temp[1]=temp[2];
temp[2]=temp[3]; temp[3]=t;
                    for(int i=0;i<4;i++) temp[i]=(byte)SBOX[temp[i]&0xFF];
                    temp[0]=(byte)((temp[0]&0xFF)^RCON[rconIter]); rconIter++;
                }
                for(int i=0;i<4;i++){
expanded[bytesGenerated]=(byte)((expanded[bytesGenerated-
16]&0xFF)^(temp[i]&0xFF)); bytesGenerated++; }
            }
            return expanded;
```

```
        }
        private static byte[] getRoundKey(byte[] expanded,int round){ byte[]
rk=new byte[16]; System.arraycopy(expanded,round*16,rk,0,16); return rk; }

        private static byte[] encryptBlock(byte[] in, byte[] expandedKey, int
blockIndex){
            byte[][] state = bytesToState(in);
            System.out.println("\n--- Encrypting block " + blockIndex + " ---
");
            System.out.println("Initial State: " + stateToHex(state));
            addRoundKey(state, getRoundKey(expandedKey,0));
            System.out.println("After AddRoundKey (Round 0): " +
stateToHex(state));
            for(int round=1; round<Nr; round++){
                subBytes(state); System.out.println("Round " + round + " -
After SubBytes: " + stateToHex(state));
                shiftRows(state); System.out.println("Round " + round + " -
After ShiftRows: " + stateToHex(state));
                mixColumns(state); System.out.println("Round " + round + " -
After MixColumns: " + stateToHex(state));
                addRoundKey(state, getRoundKey(expandedKey,round));
System.out.println("Round " + round + " - After AddRoundKey: " +
stateToHex(state));
            }
            subBytes(state); System.out.println("Round " + Nr + " - After
SubBytes: " + stateToHex(state));
            shiftRows(state); System.out.println("Round " + Nr + " - After
ShiftRows: " + stateToHex(state));
            addRoundKey(state, getRoundKey(expandedKey,Nr));
System.out.println("Round " + Nr + " - After AddRoundKey (Final): " +
stateToHex(state));
            return stateToBytes(state);
        }

        public static byte[] encrypt(byte[] plaintext, byte[] key){
            int padLen = 16 - (plaintext.length % 16); if (padLen == 0) padLen
= 16;
            byte[] padded = Arrays.copyOf(plaintext, plaintext.length +
padLen);
            for (int i = plaintext.length; i < padded.length; i++) padded[i] =
(byte) padLen;
            byte[] expandedKey = expandKey(key);
            byte[] out = new byte[padded.length]; int blocks = padded.length /
16;
            for (int i=0;i<blocks;i++){ byte[] block =
Arrays.copyOfRange(padded, i*16, (i+1)*16); byte[] enc = encryptBlock(block,
expandedKey, i); System.arraycopy(enc,0,out,i*16,16); }
```

```
            System.out.println("\nFinal Ciphertext (hex): " +
bytesToHex(out));
            return out;
        }

        private static byte[] parseKeyInput(String keyInput){ byte[] kb =
keyInput.getBytes(StandardCharsets.UTF_8); if(kb.length==16) return kb;
if(kb.length<16){ byte[] p=new byte[16]; System.arraycopy(kb,0,p,0,kb.length);
return p; } return Arrays.copyOf(kb,16); }

        public static String encryptToString(String plaintextStr, String
keyStr){
            byte[] key = parseKeyInput(keyStr);
            byte[] plaintext = plaintextStr.getBytes(StandardCharsets.UTF_8);
            System.out.println("\n--- START ENCRYPTION ---");
            byte[] ciphertext = encrypt(plaintext, key);
            return new String(ciphertext, StandardCharsets.ISO_8859_1);
        }
    }
}
```

Receiver

```
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Scanner;

public class aes_server_receiver {
    public static void main(String[] args) {
        int port = 5000;
        System.out.println("Receiver listening on port " + port);
        try (ServerSocket ss = new ServerSocket(port)) {
            try (Socket s = ss.accept();
                 DataInputStream dis = new
DataInputStream(s.getInputStream());
                 DataOutputStream dos = new
DataOutputStream(s.getOutputStream())) {

                int cipherLen = dis.readInt();
                byte[] cipherBytes = new byte[cipherLen];
                dis.readFully(cipherBytes);
```

```
                String cipherStr = new String(cipherBytes,
StandardCharsets.UTF_8);
                System.out.println("Cypher received: " + cipherStr);

                System.out.println("Ciphertext received. Please enter key to
decrypt:");
                Scanner sc = new Scanner(System.in, "UTF-8");
                String key = sc.nextLine();

                String plaintext = AES.decryptFromString(cipherStr, key);
                System.out.println("\nDecrypted plaintext: " + plaintext);

                String ack = "OK: decrypted " + plaintext.length() + " chars";
                byte[] ackBytes = ack.getBytes(StandardCharsets.UTF_8);
                dos.writeInt(ackBytes.length);
                dos.write(ackBytes);
                dos.flush();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // AES (decrypt-only)
    static class AES {
        private static final int Nr = 10;
        private static final int[] INV_SBOX = {
            0x52,0x09,0x6A,0xD5,0x30,0x36,0xA5,0x38,0xBF,0x40,0xA3,0x9E,0x81,0
xF3,0xD7,0xFB,
            0x7C,0xE3,0x39,0x82,0x9B,0x2F,0xFF,0x87,0x34,0x8E,0x43,0x44,0xC4,0
xDE,0xE9,0xCB,
            0x54,0x7B,0x94,0x32,0xA6,0xC2,0x23,0x3D,0xEE,0x4C,0x95,0x0B,0x42,0
xFA,0xC3,0x4E,
            0x08,0x2E,0xA1,0x66,0x28,0xD9,0x24,0xB2,0x76,0x5B,0xA2,0x49,0x6D,0
x8B,0xD1,0x25,
            0x72,0xF8,0xF6,0x64,0x86,0x68,0x98,0x16,0xD4,0xA4,0x5C,0xCC,0x5D,0
x65,0xB6,0x92,
            0x6C,0x70,0x48,0x50,0xFD,0xED,0xB9,0xDA,0x5E,0x15,0x46,0x57,0xA7,0
x8D,0x9D,0x84,
            0x90,0xD8,0xAB,0x00,0x8C,0xBC,0xD3,0x0A,0xF7,0xE4,0x58,0x05,0xB8,0
xB3,0x45,0x06,
            0xD0,0x2C,0x1E,0x8F,0xCA,0x3F,0x0F,0x02,0xC1,0xAF,0xBD,0x03,0x01,0
x13,0x8A,0x6B,
            0x3A,0x91,0x11,0x41,0x4F,0x67,0xDC,0xEA,0x97,0xF2,0xCF,0xCE,0xF0,0
xB4,0xE6,0x73,
            0x96,0xAC,0x74,0x22,0xE7,0xAD,0x35,0x85,0xE2,0xF9,0x37,0xE8,0x1C,0
x75,0xDF,0x6E,
```

```
            0x47,0xF1,0x1A,0x71,0x1D,0x29,0xC5,0x89,0x6F,0xB7,0x62,0x0E,0xAA,0
x18,0xBE,0x1B,
            0xFC,0x56,0x3E,0x4B,0xC6,0xD2,0x79,0x20,0x9A,0xDB,0xC0,0xFE,0x78,0
xCD,0x5A,0xF4,
            0x1F,0xDD,0xA8,0x33,0x88,0x07,0xC7,0x31,0xB1,0x12,0x10,0x59,0x27,0
x80,0xEC,0x5F,
            0x60,0x51,0x7F,0xA9,0x19,0xB5,0x4A,0x0D,0x2D,0xE5,0x7A,0x9F,0x93,0
xC9,0x9C,0xEF,
            0xA0,0xE0,0x3B,0x4D,0xAE,0x2A,0xF5,0xB0,0xC8,0xEB,0xBB,0x3C,0x83,0
x53,0x99,0x61,
            0x17,0x2B,0x04,0x7E,0xBA,0x77,0xD6,0x26,0xE1,0x69,0x14,0x63,0x55,0
x21,0x0C,0x7D
        };
        private static final int[] SBOX = { // for key expansion
            0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0
xd7,0xab,0x76,
            0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0
xa4,0x72,0xc0,
            0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0
xd8,0x31,0x15,
            0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0
x27,0xb2,0x75,
            0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0
xe3,0x2f,0x84,
            0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0
x4c,0x58,0xcf,
            0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0
x3c,0x9f,0xa8,
            0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0
xff,0xf3,0xd2,
            0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0
x5d,0x19,0x73,
            0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0
x5e,0x0b,0xdb,
            0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0
x95,0xe4,0x79,
            0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0
x7a,0xae,0x08,
            0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0
xbd,0x8b,0x8a,
            0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0
xc1,0x1d,0x9e,
            0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0
x55,0x28,0xdf,
            0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0
x54,0xbb,0x16
        };
```

```
        private static final int[] RCON =
{0x00,0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0x1B,0x36};

        private static String bytesToHex(byte[] b){ StringBuilder sb=new
StringBuilder(); for(byte x:b) sb.append(String.format("%02X", x&0xFF));
return sb.toString(); }
        private static String stateToHex(byte[][] s){ StringBuilder sb=new
StringBuilder(); for(int r=0;r<4;r++){ for(int c=0;c<4;c++)
sb.append(String.format("%02X", s[r][c]&0xFF)); if(r<3) sb.append(' ');}
return sb.toString(); }
        private static byte[][] bytesToState(byte[] in){ byte[][] s=new
byte[4][4]; for(int i=0;i<16;i++) s[i%4][i/4]=in[i]; return s; }
        private static byte[] stateToBytes(byte[][] state){ byte[] out=new
byte[16]; for(int i=0;i<16;i++) out[i]=state[i%4][i/4]; return out; }
        private static int mul(int a,int b){ int res=0; int aa=a&0xFF,
bb=b&0xFF; for(int i=0;i<8;i++){ if((bb&1)!=0) res^=aa; boolean
hi=(aa&0x80)!=0; aa=(aa<<1)&0xFF; if(hi) aa^=0x1B; bb>>>=1;} return res; }

        private static void invSubBytes(byte[][] s){ for(int r=0;r<4;r++)
for(int c=0;c<4;c++) s[r][c]=(byte)INV_SBOX[s[r][c]&0xFF]; }
        private static void invShiftRows(byte[][] s){ for(int r=1;r<4;r++){
byte[] tmp=new byte[4]; for(int c=0;c<4;c++) tmp[(c+r)%4]=s[r][c]; for(int
c=0;c<4;c++) s[r][c]=tmp[c]; } }
        private static void invMixColumns(byte[][] s){ for(int c=0;c<4;c++){
int a0=s[0][c]&0xFF,a1=s[1][c]&0xFF,a2=s[2][c]&0xFF,a3=s[3][c]&0xFF; int
r0=mul(0x0E,a0)^mul(0x0B,a1)^mul(0x0D,a2)^mul(0x09,a3); int
r1=mul(0x09,a0)^mul(0x0E,a1)^mul(0x0B,a2)^mul(0x0D,a3); int
r2=mul(0x0D,a0)^mul(0x09,a1)^mul(0x0E,a2)^mul(0x0B,a3); int
r3=mul(0x0B,a0)^mul(0x0D,a1)^mul(0x09,a2)^mul(0x0E,a3); s[0][c]=(byte)r0;
s[1][c]=(byte)r1; s[2][c]=(byte)r2; s[3][c]=(byte)r3; } }
        private static void addRoundKey(byte[][] s, byte[] rk){ for(int
c=0;c<4;c++) for(int r=0;r<4;r++)
s[r][c]=(byte)((s[r][c]&0xFF)^(rk[c*4+r]&0xFF)); }

        private static byte[] expandKey(byte[] key){
            byte[] expanded=new byte[16*(Nr+1)];
System.arraycopy(key,0,expanded,0,16);
            int bytesGenerated=16; int rconIter=1; byte[] temp=new byte[4];
            while(bytesGenerated<expanded.length){
                for(int i=0;i<4;i++) temp[i]=expanded[bytesGenerated-4+i];
                if(bytesGenerated%16==0){ byte t=temp[0]; temp[0]=temp[1];
temp[1]=temp[2]; temp[2]=temp[3]; temp[3]=t; for(int i=0;i<4;i++)
temp[i]=(byte)SBOX[temp[i]&0xFF];
temp[0]=(byte)((temp[0]&0xFF)^RCON[rconIter]); rconIter++; }
                for(int i=0;i<4;i++){
expanded[bytesGenerated]=(byte)((expanded[bytesGenerated-
16]&0xFF)^(temp[i]&0xFF)); bytesGenerated++; }
            }
```

```
            return expanded;
        }
        private static byte[] getRoundKey(byte[] expanded,int round){ byte[]
rk=new byte[16]; System.arraycopy(expanded,round*16,rk,0,16); return rk; }

        private static byte[] decryptBlock(byte[] in, byte[] expandedKey, int
blockIndex){
            byte[][] state = bytesToState(in);
            System.out.println("\n--- Decrypting block " + blockIndex + " ---
");
            System.out.println("Initial State (ciphertext block): " +
stateToHex(state));
            addRoundKey(state, getRoundKey(expandedKey, Nr));
            System.out.println("After AddRoundKey (Round " + Nr + "): " +
stateToHex(state));
            for (int round = Nr-1; round >= 1; round--) {
                invShiftRows(state); System.out.println("Round " + round + " -
After InvShiftRows: " + stateToHex(state));
                invSubBytes(state); System.out.println("Round " + round + " -
After InvSubBytes: " + stateToHex(state));
                addRoundKey(state, getRoundKey(expandedKey, round));
System.out.println("Round " + round + " - After AddRoundKey: " +
stateToHex(state));
                invMixColumns(state); System.out.println("Round " + round + "
- After InvMixColumns: " + stateToHex(state));
            }
            invShiftRows(state); System.out.println("Round 0 - After
InvShiftRows: " + stateToHex(state));
            invSubBytes(state); System.out.println("Round 0 - After
InvSubBytes: " + stateToHex(state));
            addRoundKey(state, getRoundKey(expandedKey, 0));
System.out.println("Round 0 - After AddRoundKey (Recovered Plaintext Block): "
+ stateToHex(state));
            return stateToBytes(state);
        }

        public static byte[] decrypt(byte[] ciphertext, byte[] key) {
            if (ciphertext.length % 16 != 0) throw new
IllegalArgumentException("Ciphertext length must be multiple of 16 bytes");
            byte[] expandedKey = expandKey(key);
            byte[] out = new byte[ciphertext.length];
            int blocks = ciphertext.length / 16;
            for (int i = 0; i < blocks; i++) {
                byte[] block = Arrays.copyOfRange(ciphertext, i * 16, (i+1) *
16);
                byte[] dec = decryptBlock(block, expandedKey, i);
                System.arraycopy(dec, 0, out, i*16, 16);
            }
```

```
            int padLen = out[out.length - 1] & 0xFF;
            if (padLen < 1 || padLen > 16) return out;
            for (int i = out.length - padLen; i < out.length; i++) if ((out[i]
& 0xFF) != padLen) return out;
            return Arrays.copyOf(out, out.length - padLen);
        }

        private static byte[] parseKeyInput(String keyInput){ byte[] kb =
keyInput.getBytes(StandardCharsets.UTF_8); if(kb.length==16) return kb;
if(kb.length<16){ byte[] p=new byte[16]; System.arraycopy(kb,0,p,0,kb.length);
return p; } return Arrays.copyOf(kb,16); }

        public static String decryptFromString(String cipherString, String
keyStr){
            byte[] key = parseKeyInput(keyStr);
            byte[] ciphertext =
cipherString.getBytes(StandardCharsets.ISO_8859_1);
            byte[] recovered = decrypt(ciphertext, key);
            return new String(recovered, StandardCharsets.UTF_8);
        }
    }
}
```

Output [screenshot]

Sender

```
PS E:\aa vit sem6\z_crypto-lab> javac aes_client_sender.java
PS E:\aa vit sem6\z_crypto-lab> java aes_client_sender
Plaintext: ArkitaBarua
Key: Security1099Key

--- START ENCRYPTION ---

--- Encrypting block 0 ---
Initial State: 41747205 72617505 6B426105 69610505
After AddRoundKey (Round 0): 1206434E 17084560 0836587C 1C183C05
Round 1 - After SubBytes: C96F1A2F F0306ED0 30056A10 9CADEB6B
Round 1 - After ShiftRows: C96F1A2F 306ED0F0 6A103005 6B9CADEB
Round 1 - After MixColumns: D8E0C2BB 7C1F5C30 909E46F3 CCEC8F49
Round 1 - After AddRoundKey: C78D9EAC AFA5D6DF 90EA0BC7 0A5309CF
Round 2 - After SubBytes: C65D0B91 7906F69E 60872BC6 67ED018A
Round 2 - After ShiftRows: C65D0B91 06F69E79 2BC66087 8A67ED01
Round 2 - After MixColumns: 3C1A2234 3D9C61F0 139579FE 73192254
Round 2 - After AddRoundKey: FEB5D1D0 F6ED9AE4 57A504B7 45902DDD
Round 3 - After SubBytes: BBD53E70 4255B869 5B06F2A9 6E60D8C1
Round 3 - After ShiftRows: BBD53E70 55B86942 F2A95B06 C16E60D8
Round 3 - After MixColumns: A1A5FCF8 DD306126 4996414D E8A9B07F
Round 3 - After AddRoundKey: 9D369C7C 2DB11B48 AA45EFAA B77F692F
Round 4 - After SubBytes: 5E05DE10 D8C8AF52 AC6EDFAC A9D2F915
Round 4 - After ShiftRows: 5E05DE10 C8AF52D8 DFACAC6E 15A9D2F9
Round 4 - After MixColumns: 35E52FC4 BA0647F0 0C09A204 DF45386F
```

```
Round 3 - After AddRoundKey: 9D369C7C 2DB11B48 AA45EFAA B77F692F
Round 4 - After SubBytes: 5E05DE10 D8C8AF52 AC6EDFAC A9D2F915
Round 4 - After ShiftRows: 5E05DE10 C8AF52D8 DFACAC6E 15A9D2F9
Round 4 - After MixColumns: 35E52FC4 BA0647F0 0C09A204 DF45386F
Round 4 - After AddRoundKey: 9EDD7718 DEE3D801 BC6A6F2E DF933730
Round 5 - After SubBytes: 0BC1F5AD 1D11617C 6502A831 9EDC9A04
Round 5 - After ShiftRows: 0BC1F5AD 11617C1D A8316502 049EDC9A
Round 5 - After MixColumns: 8995CCFE CECE7E0B 5D7B3C01 AC2FBEDC
Round 5 - After AddRoundKey: 93B7B658 4FAA8501 2267EDFA 2A7FE1DC
Round 6 - After SubBytes: DCA94E6A 84AC977C 9385552D E5D2F886
Round 6 - After ShiftRows: DCA94E6A AC977C84 552D9385 86E5D2F8
Round 6 - After MixColumns: 9F23593E E60ECA15 4B5062EC 918B8254
Round 6 - After AddRoundKey: C25C5C9D 68E4DB0E 5750B3C6 33792FF9
Round 7 - After SubBytes: 254A4A5E 4569B9AB 5B536DB4 C3B61599
Round 7 - After ShiftRows: 254A4A5E 69B9AB45 6DB45B53 99C3B615
Round 7 - After MixColumns: 05339F35 D9275C34 26DE9682 424E59DE
Round 7 - After AddRoundKey: B7FE575E B2A6CCBF AF57CEF0 EA14AE84
Round 8 - After SubBytes: A9BB5B58 37244B08 795B8B8C 87FAE45F
Round 8 - After ShiftRows: A9BB5B58 244B0837 8B8C795B 5F87FAE4
Round 8 - After MixColumns: F1BB2D56 38253A3F 6161B4EE F1047357
Round 8 - After AddRoundKey: FE792737 138F008E 56DF527A 26890977
Round 9 - After SubBytes: BBB6CC9A 7D736319 B19E00DA F7A701F5
Round 9 - After ShiftRows: BBB6CC9A 7363197D 00DAB19E F5F7A701
Round 9 - After MixColumns: 0DFFBE37 A8F291D8 CC785EC3 548DB254
Round 9 - After AddRoundKey: D1E1AA42 A15108F0 4C46868F 6C387DBB
Round 10 - After SubBytes: 3EF8AC2C 32D1308C 295A4473 5007FFEA
Round 10 - After ShiftRows: 3EF8AC2C D1308C32 4473295A EA5007FF
Round 10 - After AddRoundKey (Final): E038788D F1B39600 1B1290AF 4F40D8CF

Final Ciphertext (hex): E0F11B4F38B31240789690D88D00AFCF
Cypher text sent: àñ8?@x??????
Receiver response: OK: decrypted 11 chars
```

Receiver

```
PS E:\aa vit sem6\z_crypto-lab> javac aes_server_receiver.java
PS E:\aa vit sem6\z_crypto-lab> java aes_server_receiver
Receiver listening on port 5000
Cypher received: àñ8?@x??????
Ciphertext received. Please enter key to decrypt:
Security1099Key

--- Decrypting block 0 ---
Initial State (ciphertext block): E038788D F1B39600 1B1290AF 4F40D8CF
After AddRoundKey (Round 10): 3EF8AC2C D1308C32 4473295A EA5007FF
Round 9 - After InvShiftRows: 3EF8AC2C 32D1308C 295A4473 5007FFEA
Round 9 - After InvSubBytes: D1E1AA42 A15108F0 4C46868F 6C387DBB
Round 9 - After AddRoundKey: 0DFFBE37 A8F291D8 CC785EC3 548DB254
Round 9 - After InvMixColumns: BBB6CC9A 7363197D 00DAB19E F5F7A701
Round 8 - After InvShiftRows: BBB6CC9A 7D736319 B19E00DA F7A701F5
Round 8 - After InvSubBytes: FE792737 138F008E 56DF527A 26890977
Round 8 - After AddRoundKey: F1BB2D56 38253A3F 6161B4EE F1047357
Round 8 - After InvMixColumns: A9BB5B58 244B0837 8B8C795B 5F87FAE4
Round 7 - After InvShiftRows: A9BB5B58 37244B08 795B8B8C 87FAE45F
Round 7 - After InvSubBytes: B7FE575E B2A6CCBF AF57CEF0 EA14AE84
Round 7 - After AddRoundKey: 05339F35 D9275C34 26DE9682 424E59DE
Round 7 - After InvMixColumns: 254A4A5E 69B9AB45 6DB45B53 99C3B615
Round 6 - After InvShiftRows: 254A4A5E 4569B9AB 5B536DB4 C3B61599
Round 6 - After InvSubBytes: C25C5C9D 68E4DB0E 5750B3C6 33792FF9
Round 6 - After AddRoundKey: 9F23593E E60ECA15 4B5062EC 918B8254
Round 6 - After InvMixColumns: DCA94E6A AC977C84 552D9385 86E5D2F8
Round 5 - After InvShiftRows: DCA94E6A 84AC977C 9385552D E5D2F886
Round 5 - After InvSubBytes: 93B7B658 4FAA8501 2267EDFA 2A7FE1DC
Round 5 - After AddRoundKey: 8995CCFE CECE7E0B 5D7B3C01 AC2FBEDC
Round 5 - After InvMixColumns: 0BC1F5AD 11617C1D A8316502 049EDC9A
Round 4 - After InvShiftRows: 0BC1F5AD 1D11617C 6502A831 9EDC9A04
Round 4 - After InvSubBytes: 9EDD7718 DEE3D801 BC6A6F2E DF933730
Round 4 - After AddRoundKey: 35E52FC4 BA0647F0 0C09A204 DF45386F
```

```
Round 5 - After InvMixColumns: 0BC1F5AD 11617C1D A8316502 049EDC9A
Round 4 - After InvShiftRows: 0BC1F5AD 1D11617C 6502A831 9EDC9A04
Round 4 - After InvSubBytes: 9EDD7718 DEE3D801 BC6A6F2E DF933730
Round 4 - After AddRoundKey: 35E52FC4 BA0647F0 0C09A204 DF45386F
Round 4 - After InvMixColumns: 5E05DE10 C8AF52D8 DFACAC6E 15A9D2F9
Round 3 - After InvShiftRows: 5E05DE10 D8C8AF52 AC6EDFAC A9D2F915
Round 3 - After InvSubBytes: 9D369C7C 2DB11B48 AA45EFAA B77F692F
Round 3 - After AddRoundKey: A1A5FCF8 DD306126 4996414D E8A9B07F
Round 3 - After InvMixColumns: BBD53E70 55B86942 F2A95B06 C16E60D8
Round 2 - After InvShiftRows: BBD53E70 4255B869 5B06F2A9 6E60D8C1
Round 2 - After InvSubBytes: FEB5D1D0 F6ED9AE4 57A504B7 45902DDD
Round 2 - After AddRoundKey: 3C1A2234 3D9C61F0 139579FE 73192254
Round 2 - After InvMixColumns: C65D0B91 06F69E79 2BC66087 8A67ED01
Round 1 - After InvShiftRows: C65D0B91 7906F69E 60872BC6 67ED018A
Round 1 - After InvSubBytes: C78D9EAC AFA5D6DF 90EA0BC7 0A5309CF
Round 1 - After AddRoundKey: D8E0C2BB 7C1F5C30 909E46F3 CCEC8F49
Round 1 - After InvMixColumns: C96F1A2F 306ED0F0 6A103005 6B9CADEB
Round 0 - After InvShiftRows: C96F1A2F F0306ED0 30056A10 9CADEB6B
Round 0 - After InvSubBytes: 1206434E 17084560 0836587C 1C183C05
Round 0 - After AddRoundKey (Recovered Plaintext Block): 41747205 72617505 6B426105 69610505

Decrypted plaintext: ArkitaBarua
PS E:\aa vit sem6\z_crypto-lab>
```